Optimisation pour l'Apprentissage Automatique

IASD Apprentissage

Cours 1

October 1, 2026

Shuyu Dong

Dauphine | PSL UNIVERSITÉ PARIS LAMSADE UMR 7243

Programme

- Introduction et méthodes de gradient
- Optimiseurs · Calcul de dérivées
- Différentiation automatique
- Problèmes convexes et non-convexes
- Gradients, sous-gradients · Méthodes proximales
- Méthodes de gradient et programmation linéaire

Evaluation :

- Examen écrit final
- 1 Projet final

Réferences :

- Supports du cours *Optimisation pour l'apprentissage automatique* de Clément Royer.
  URL https://www.lamsade.dauphine.fr/~croyer/coursOAA.html.
- (Wright and Ma, 2022): High-dimensional data analysis with low-dimensional models: Principles, computation, and applications. Cambridge University Press.
  URL https://book-wright-ma.github.io/.
- Autres réferences à détailler

Plan du cours

- 1.1 – Descente de gradient
- 1.2 – Exercices 1

Plan du cours

1.1 – Descente de gradient

1.2 – Exercices 1

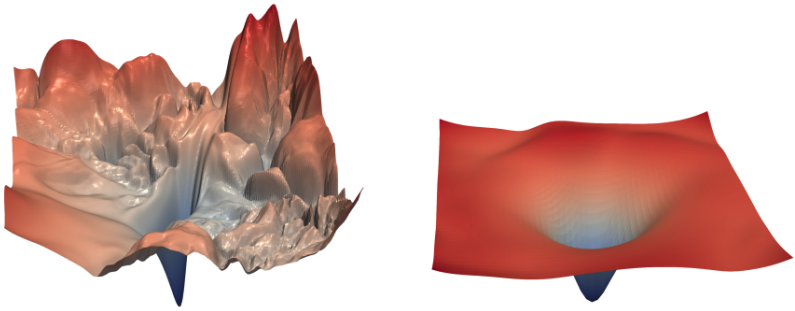
2.1 – Gradient stochastique

2.2 – Méthodes avanceées de gradient stochastique (Optimiseurs)

2.3 – TD

Optimisation pour l'apprentissage automatique

Exemple :



(a) without skip connections

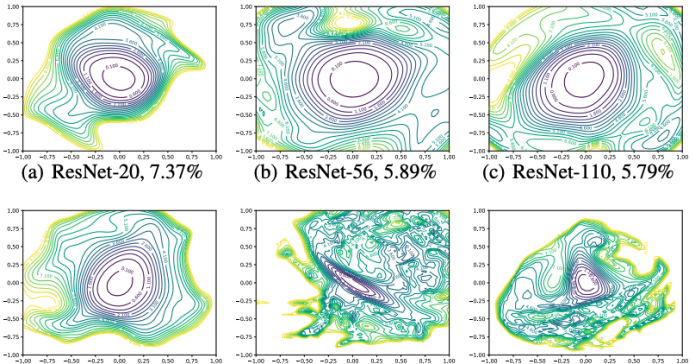
(b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

(Li et al., 2018)

Optimisation pour l'apprentissage automatique

Exemple :



(a) ResNet-20, 7.37% (b) ResNet-56, 5.89% (c) ResNet-110, 5.79%

(d) ResNet-20-NS, 8.18% (e) ResNet-56-NS, 13.31% (f) ResNet-110-NS, 16.44%

Figure 5: 2D visualization of the loss surface of ResNet and ResNet-noshort with different depth.

(Li et al., 2018)

Pourquoi des gradients ?

La descente de gradient est à l'origine de la plupart des algorithmes d'optimisation pour l'apprentissage, par exemple, des méthodes pour optimiser des réseaux de neurones.

Problème :

$$\underset{w\in\mathbb{R}^d}{\text{minimiser}} \quad f(w) \qquad \text{où} \qquad f:\mathbb{R}^d \to \mathbb{R} \quad \text{est de classe } C^1. \tag{1}$$

Les **variables / paramètres** sur lesquels on optimise sont

$$w = (w_1, \dots, w_d)^{\mathrm{T}} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \qquad w_i \in \mathbb{R}.$$

Pour tout $w \in \mathbb{R}^d$, la variation de f autour de w est décrite par le vecteur gradient de f en w, noté

$$\nabla f(w) \in \mathbb{R}^d.$$

Pourquoi des gradients ? (cont.)

Proposition : *Toute solution du problème d'optimisation* (1) *est un point en lequel le gradient est nul. C'est à dire :*

$$w^\star \in \arg\min_{w \in \mathbb{R}^d} f(w) \qquad \Longrightarrow \qquad \nabla f(w^\star) = 0.$$

Notation : on utilise "$\nabla f(w^\star) = 0$" pour l'expression $\nabla f(w^\star) = (0, \dots, 0)^{\mathrm{T}} \in \mathbb{R}^d$.

Preuve : Si le gradient $\nabla f(w)$ n'est pas nul, alors il existe un point v tel que $f(v) < f(w)$; il suffit de faire un petit déplacement (depuis w) dans la direction opposée du gradient $\nabla f(w)$ pour en trouver un tel point v. □

L'ensemble des solutions (ou minima globaux) du problème est donc contenu dans l'ensemble des points où le gradient est nul.

Remarque : Il peut exister des points en lesquels le gradient est nul mais qui ne sont pas des solutions du problème.

Exemple : pour $d = 1$, $f(w) = \frac{1}{2}(w^2 - 1)^2$.

- Deux minima globaux (sur lesquels le gradient est nul)
- Mais il y un point non-optimal sur lequel le gradient s'annule aussi.

Algorithme de descente de gradient

Algorithm 1 Descente de gradient

Initialisation : Choisir w_0
for $k = 0, 1, 2, \ldots$
    Calculer le gradient $\nabla f(w_k)$
    Calculer $w_{k+1} = w_k - \alpha_k \nabla f(w_k)$ avec $\alpha_k > 0$
end for

Questions :

- Est-ce que cet algorithme fonctionne ?
- Comment l'implémenter (quelle initialisation, choix de α_k, critère d'arrêt, ...) ?

Algorithme de descente de gradient (cont.)

Exemple : Descente de gradient et la régression linéaire.
Etant donnée les échantillons $\{x_i, y_i\}_{i=1,\dots,m}$ où $x_i \in \mathbb{R}^d$ et $y_i \in \mathbb{R}$, on veut minimiser :

$$f(w) = \frac{1}{2m}\|Xw - y\|_2^2 = \frac{1}{2m}\sum_{i=1}^{m}({x_i}^{\mathrm{T}} w - y_i)^2,$$

avec $X = (x_1, \dots, x_m)^{\mathrm{T}} \in \mathbb{R}^{m\times d}$ et $w \in \mathbb{R}^d$.

- f est différentiable et son gradient vaut :

$$\nabla f(w) = \frac{1}{m}X^{\mathrm{T}}(Xw - y).$$

- On peut appliquer la descente de gradient à ce problème :

$$w_{k+1} = w_k - \frac{\alpha_k}{m}X^{\mathrm{T}}(Xw_k - y)$$

Notation : pour tout $u, v \in \mathbb{R}^n$, on note $u^{\mathrm{T}}v := \sum_{i=1}^n u_i v_i$ pour le produit scalaire, et $\|u\|_2^2 := \sum_{i=1}^n u_i^2$ pour la norme ℓ_2.

Remarque :

- Si $\nabla f(w_k) = 0$, alors $w_{k+1} = w_k - \alpha_k \nabla f(w_k) = w_k - 0 = w_k$.

Algorithme de descente de gradient (cont.)

- L'itération se réécrit :

$$w_{k+1} = \left[I - \frac{\alpha_k}{m} X^{\mathrm{T}} X \right] w_k + \frac{\alpha_k}{m} X^{\mathrm{T}} y$$

- En combinant cette formule sur plusieurs itérations, on peut garantir (sous certaines hypothèses) que la méthode converge vers une solution du problème :
  1. Soit on atteint un point fixe.
  2. Soit $\lim_{k \to \infty} \{\omega_k\} = w^*$ qui minimise $f(w)$.
- En pratique, on cherche à converger le plus rapidement possible dans un budget imparti. La descente de gradient garantit une vitesse de convergence en un nombre d'itérations fixé.

Théorème (Convergence de la descente de gradient pour la régression linéaire) : *Supposons qu'il existe $w^* \in \mathbb{R}^d$ tel que $Xw^* = y$, et que $X^{\mathrm{T}}X$ est définie positive. On utilise la descente de gradient (Algorithm 1) avec $\alpha_k = \frac{m}{\|X^{\mathrm{T}}X\|}$. Alors, la suite des itérés $\{w_k\}_{k \geq 0}$ de la descente de gradient vérifie :*

$$\|w_k - w^*\|_2^2 \leq \rho^k \|w_0 - w^*\|_2^2 \quad \textit{avec } \rho \textit{ telle que } 0 \leq \rho < 1.$$

Preuve : (à voir au tableau)

Algorithme de descente de gradient (cont.)

Par conséquent, on a alors $\lim_{k\to\infty} \|w_k - w^*\|_2 = 0$, et on dit que la descente de gradient converge sur ce problème en **vitesse linéaire**.

Remarque : Si w^* satisfait $Xw^* = y$, alors :

$$\nabla f(w^*) = \frac{1}{m} X^T (Xw^* - y) = 0 \in \mathbb{R}^d$$

et

$$f(w) = \frac{1}{2m} \|Xw - y\|_2^2 \geq 0 = \frac{1}{2m} \|Xw^* - y\|_2^2 = f(w^*) \quad \forall w \in \mathbb{R}^d$$

Ainsi, $f(w) \geq f(w^*)$, $\forall w \in \mathbb{R}^d$, i.e., $w^* \in \arg\min_{w\in\mathbb{R}^d} f(w)$.

Convergence linéaire au-delà de la régression linéaire

Pour un problème

$$\underset{w\in\mathbb{R}^d}{\text{minimise}}\ f(w),\ \text{avec } f \textit{ fortement} \text{ convexe},$$

on peut montrer que $\|w_k - w^*\|$ décroît à une vitesse linéaire, où w_k est obtenu par k itérations d'une descente de gradient.

Definition : f est de classe $C_L^{1,1}$ si $f \in C^1$ et ∇f est L-lipschitzien.

Remarque :
- ∇f est L-lipschitzien $\iff \forall (v,w) \in \mathbb{R}^d \times \mathbb{R}^d, \quad \|\nabla f(v) - \nabla f(w)\| \leq L\|v - w\|$.
- Lorsque v est proche de w, $\nabla f(v)$ est proche de $\nabla f(w)$.

Les résultats pour la régression linéaire s'étendent à toute fonction $C_L^{1,1}$ et fortement convexe.

Definition :

- $f : \mathbb{R}^d \to \mathbb{R}$ de classe C^1 est **convexe** si et seulement si :

$$\forall (v,w) \in \mathbb{R}^d \times \mathbb{R}^d, \quad f(v) \geq f(w) + \nabla f(w)^T (v - w).$$

Convergence linéaire au-delà de la régression linéaire (cont.)

- $f : \mathbb{R}^d \to \mathbb{R}$ de classe C^1 est μ-**fortement convexe** ($\mu > 0$) si et seulement si :

$$\forall (v, w) \in \mathbb{R}^d \times \mathbb{R}^d, \quad f(v) \geq f(w) + \nabla f(w)^T (v - w) + \frac{\mu}{2} \|v - w\|_2^2.$$

Exemple : La régression linéaire est définie via une fonction convexe, qui est fortement convexe lorsque $X^{\mathrm{T}} X$ est définie positive (auquel cas $\mu = \sigma_{\min}(X^T X) > 0$).

Exemple : Régression logistique

$$\underset{w \in \mathbb{R}^d}{\text{minimiser}} \ \frac{1}{m} \sum_{i=1}^{m} \log \left(1 + e^{-y_i x_i^T w}\right) + \frac{\lambda}{2} \|w\|_2^2$$

avec $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$, $\lambda \geq 0$.

- La fonction objectif est toujours convexe et elle est λ-fortement convexe si $\lambda > 0$.
- Elle est $C_L^{1,1}$ avec $L = \frac{4\|X^T X\|}{m} + \lambda$.

Propriétés des fonctions (fortement) convexes

Proposition :

- *Si f est C^1 et convexe, alors $w^* \in \arg\min_w f(w)$ si et seulement si $\nabla f(w^*) = 0$.*
- *Si f est C^1 et fortement convexe, alors il existe une unique solution au problème ($\arg\min_w f(w)$ contient un seul élément).*

Exemple :

$$f(w) = \frac{1}{4}\|Xw - y\|_2^2, \quad d = 2, \quad X = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

f est convexe (pas fortement convexe).

$$\arg\min_w f(w) = \left\{(1, w_2)^{\mathrm{T}} : w_2 \in \mathbb{R}\right\}$$

Exemple :

$$\hat{f}(w) = \frac{1}{4}\|\hat{X}w - y\|_2^2, \quad \hat{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$\hat{f}$ est fortement convexe.

$$\arg\min_w \hat{f}(w) = \left\{(1, 1)^{\mathrm{T}}\right\}$$

Propriétés des fonctions (fortement) convexes (cont.)

Théorème : *Soit* $f : \mathbb{R}^d \to \mathbb{R}$ *une fonction* $C_L^{1,1}$ *convexe et soit* $\{w_k\}_{k\geq 0}$ *la suite des itérés de la descente de gradient avec* $\alpha_k = \frac{1}{L}$. *Alors :*

$$f(w_k) - \min_{w\in\mathbb{R}^d} f(w) \leq \mathcal{O}\left(\frac{1}{k}\right).$$

Notes :

- $\min_{w\in\mathbb{R}^d} f(w)$ est la valeur optimale du problème.
- $\mathcal{O}(A)$: une quantité entre $c_1 A$ et $c_2 A$ pour deux constants $0 < c_1 \leq c_2$.
- Ici, $\mathcal{O}\left(\frac{1}{k}\right) = \frac{L}{2k} \cdot \text{dist}^2\left(w_0, \arg\min_w f(w)\right)$.

On dit que la descente de gradient converge en $\mathcal{O}(\frac{1}{k})$ dans le cas convexe.

Propriétés des fonctions (fortement) convexes (cont.)

Théorème : *Si de plus f est μ-fortement convexe, alors :*

$$f(w_k) - \min_{w \in \mathbb{R}^d} f(w) \leq \mathcal{O}\left(\left(1 - \frac{\mu}{L}\right)^k\right).$$

Remarque :
Le rapport $\kappa = \frac{L}{\mu} \geq 1$ est appelé le conditionnement du problème.

- Plus κ est proche de 1, plus la convergence est rapide.
- Plus κ est grand, plus le problème est dit mal conditionné, et la convergence ralentit.

La descente de gradient converge en $\mathcal{O}(\rho^k)$ (ou en vitesse linéaire) dans le cas fortement convexe.

- Une meilleure vitesse de convergence que dans le cas convexe ($\rho^k \to 0$ plus vite que $\frac{1}{k}$).

Bilan

- Le gradient (quand il existe) vaut 0 en tout minimum global.
  - Si la fonction est convexe, les seuls points où le gradient vaut 0 sont les minima globaux.
  - Si la fonction est fortement convexe, il existe un unique minimum global.
- L'algorithme de descente de gradient s'applique pour tout problème où la fonction à minimiser est C^1.
  - Si elle est $C_L^{1,1}$, on peut utiliser $\alpha_k = \frac{1}{L}$ dans l'algorithme.
  - Cela garantit une vitesse de convergence, notamment dans les cas convexe et fortement convexe.

Plan du cours

1.1 – Descente de gradient

1.2 – Exercices 1

2.1 – Gradient stochastique

2.2 – Méthodes avanceées de gradient stochastique (Optimiseurs)

2.3 – TD

Optimisation pour l'Apprentissage Automatique

IASD Apprentissage

Cours 2

November 10, 2026

Shuyu Dong

Dauphine | PSL UNIVERSITÉ PARIS LAMSADE UMR 7243

Plan du cours

- 2.1 – Gradient stochastique
- 2.2 – Méthodes avanceées de gradient stochastique (Optimiseurs)
- 2.3 – TD

Plan du cours

1.1 – Descente de gradient

1.2 – Exercices 1

2.1 – Gradient stochastique

2.2 – Méthodes avanceées de gradient stochastique (Optimiseurs)

2.3 – TD

Méthodes de gradient stochastique

Problème d'optimisation d'**une somme finie de fonctions** :

$$\min_{w \in \mathbb{R}^d} f(w) := \frac{1}{n} \sum_{i=1}^{n} f_i(w),$$

où $f_i \in \mathcal{C}^1$ dépend du i-ème point du jeu de données.

En apprentissage supervisé : $f_i(w) = \ell(h(w; x_i), y_i)$ pour un jeu de données $(x_i, y_i)_{i=1}^n$, où

- $h(w; \cdot) : \mathcal{X} \to \mathcal{Y}$ est une classe de fonction de prédiction parametrée par $w \in \mathbb{R}^d$, et
- $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ est une fonction de perte.

Exemple 1 : Régression linéaire

$$f(w) = \frac{1}{2n} \sum_{i=1}^{n} (w^{\mathrm{T}} x_i - y_i)^2 = \frac{1}{2n} \|Xw - y\|^2 \quad \Rightarrow$$

$$f_i(w) = \frac{1}{2} (w^{\mathrm{T}} x_i - y_i)^2.$$

Méthodes de gradient stochastique

Descente de gradient :

$$w_{k+1} = w_k - \alpha_k \nabla f(w_k), \text{ avec } \alpha_k > 0 \tag{GD}$$

Gradient stochastique :

$$w_{k+1} = w_k - \alpha_k \nabla f_{i_k}(w_k), \text{ avec } \alpha_k > 0 \tag{SGD}$$

où i_k est tiré aléatoirement dans $\{1, \ldots, n\}$.

Algorithm 2 Descente de gradient stochastique

1: Initialize $w_0 \in \mathbb{R}^d$
2: **for** $t = 1, \ldots$ **do**
3: Déterminer un pas $\alpha_k > 0$
4: Tirer aléatoirement un index $i_k \in \{1, \ldots, n\}$
5: Calculer $w_{k+1} \leftarrow w_k - \alpha_k \nabla f_{i_k}(w_k)$
6: **end for**

Méthodes avancées de gradient stochastique

Note (Exemple 4.2.1, echantillon uniforme): si l'indexe i_k est tiré uniformément dans $\{1, \ldots, n\}$, alors Algorithm 2 satisfait "Assumption 4.2.2":

- $\mathbb{E}_{i_k}[\nabla f_{i_k}(w_k)] = \nabla f(w_k)$;
- $\mathbb{E}_{i_k}[\|\nabla f_{i_k}(w_k)\|^2] \leq \sigma^2 + \|\nabla f(w_k)\|_2^2$ with $\sigma^2 > 0$.

  Exemple : i_k est tiré uniformément au hasard dans $\{1, \ldots, n\}$. En supposant i_k indépendant de w_k, avec $P(i_k = j) = \frac{1}{n}$ pour tout $j \in \{1, \ldots, n\}$: alors

$$\mathbb{E}_{i_k}[\nabla f_{i_k}(w_k)] = \sum_{j=1}^{n} P(i_k = j)\, \nabla f_j(w_k) = \frac{1}{n} \sum_{j=1}^{n} \nabla f_j(w_k) = \nabla f(w_k).$$

En moyenne, une itération de gradient stochastique correspond à une itération de descente de gradient.

Méthodes de gradient stochastique

Gradient stochastique **par batch** (ou fournées / paquets)
Idée : Prendre plusieurs indices par itération

$$w_{k+1} := w_k - \alpha_k \cdot \left(\frac{1}{|S_k|} \sum_{i \in S_k} \nabla f_i(w_k) \right) \qquad \text{(Batch SGD)}$$

où $|S_k|$ est le nombre d'éléments du sous-ensemble d'indices $S_k \subseteq \{1, \ldots, n\}$ tirés aléatoirement.

Deux cas particuliers :

1. *Tirage avec ou sans remise :*
   Dans certaines références, le tirage peut contenir ou non le même indice plusieurs fois.
2. *Pour une taille de batch* $|S_k| = 1$ *:*
   $\frac{1}{|S_k|} \sum_{i \in S_k} \nabla f_i(w_k) = \nabla f_{i_k}(w_k)$ (SGD standard)
3. *Pour une taille de batch* $|S_k| = n$ $(S_k = \{1, \ldots, n\})$:
   $\frac{1}{n} \sum_{i=1}^{n} \nabla f_i(w_k) = \nabla f(w_k)$ (GD)

Descente de gradient stochastique

Deux régimes pour batch SG

Mini-batch : $1 < |S_k| \ll m$

- Il peut converger plus rapidement que le gradient stochastique classique (SGD) dans les premières itérations.
- Il peut osciller autour d'une valeur plus faible que pour le SGD.

Large batch : $|S_k| \gg 1$

- Moins d'oscillations (que mini-batch) en général.
- Convergence plus lente dans les premières itérations.
- Convergence vers des valeurs plus faibles que le SGD.

En pratique : On verra souvent $|S_k| = 32, 64, 128, 256$, ce qui correspond au nombre de processeurs que l'on peut utiliser pour calculer les gradients en parallèle.

Méthodes de gradient stochastique

Algorithm 3 Descente de gradient stochastique par batch (Batch SGD)

1: Initialize $w_0 \in \mathbb{R}^d$
2: **for** $t = 1, \ldots$ **do**
3: Déterminer un pas $\alpha_k > 0$
4: Tirer aléatoirement un sous-ensemble S_k d'indices $S_k \subseteq \{1, \ldots, n\}$
5: Calculer $w_{k+1} \leftarrow w_k - \alpha_k \frac{1}{|S_k|} \sum_{j \in S_k} \nabla f_{i_k}(w_k)$
6: **end for**

Implémentation du gradient stochastique (Conseils)

- **Pour le pas α :** Chercher la plus grande valeur cohérente pour laquelle l'algorithme ne diverge pas. (Références : M. Hardt & B. Recht).
- **Pour $|S_k|$:** Choix en fonction des capacités de parallélisation.
- Faire tourner l'algorithme jusqu'à stagnation (de préférence sur les données test), puis réduire α (ex : $\alpha \leftarrow \alpha/10$).
- Continuer jusqu'à ce que le budget d'époques (*epochs*) soit épuisé.

Méthodes de gradient stochastique (cont.)

Époque
Si on affiche une comparaison entre une méthode stochastique et la descente de gradient pour le **même nombre d'itérations**, la descente de gradient paraît meilleure.

Cependant, une comparaison en termes d'itérations occulte le fait que **les itérations n'ont pas le même coût en termes d'accès aux données** :

- **1 itération de GD** coûte un parcours complet de tout le jeu de données (m éléments, soit m accès).
- **1 itération de SG** coûte 1 seul accès (ou $|S_k|$ accès pour un mini-batch).

Si l'on utilise des mini-batches de taille b, une époque contient environ

$$K_{\text{epoch}} = \left\lceil \frac{n}{b} \right\rceil$$

itérations de mini-batch.

Méthodes de gradient stochastique

Définition : Une **epoch (époque)** correspond à un coût équivalent à un parcours complet du jeu de données (soit m accès aux éléments).

- 1 epoch = 1 itération de GD.
- 1 epoch = m itérations de SG (avec batch = 1).

$\Longrightarrow$ Pour un même budget d'époques (souvent bien plus petit que le nombre de points m dans le jeu de données), on peut effectuer en fait **beaucoup plus d'itérations de gradient stochastique**, ce qui conduit à une convergence effective plus rapide.

Remarque : On peut étudier les vitesses de convergence à la fois en fonction du nombre d'itérations et en fonction du nombre d'époques (notamment pour des fonctions convexes).

(TD) f_i convexes.

	GD	SGD
K itérations	$O\left(\frac{1}{K}\right)$	$O\left(\frac{1}{\sqrt{K}}\right)$
K épochs	$O\left(\frac{1}{K}\right)$	$O\left(\frac{1}{\sqrt{nK}}\right) \ll O\left(\frac{1}{K}\right)$ quand $n \gg K$

Méthodes de gradient stochastique

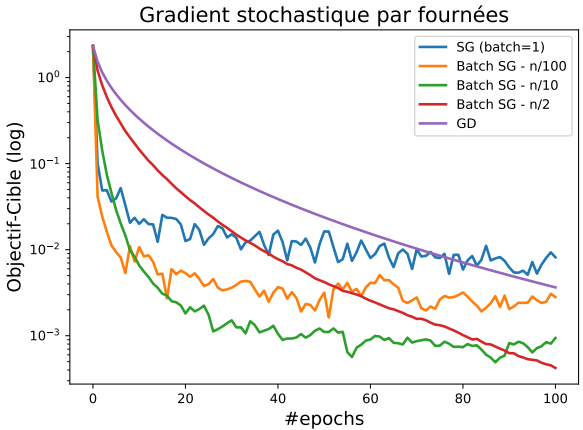


Figure: Évolution de $\log(f(w_k) - \min f(w))$ en fonction du nombre d'époques (#epochs) pour $n = 1000$.

Méthodes de gradient stochastique (cont.)

Observations sur les courbes ($n = 1000$) :

- **SG (batch $= 1$)** : Convergence rapide au début mais très bruitée/oscillante avec un plateau autour de 10^{-2} (effectue 100 000 itérations de SG).
- **Batch SG** - $n/100$: Meilleure stabilité que SG pur avec un compromis vitesse/bruit.
- **Batch SG** - $n/10$: Convergence fluide et rapide vers des valeurs cibles plus basses.
- **Batch SG** - $n/2$: Proche de la descente de gradient déterministe.
- **GD** : Décroissance régulière et lisse (effectue 100 itérations complètes de GD pour 100 époques).

Plan du cours

1.1 – Descente de gradient

1.2 – Exercices 1

2.1 – Gradient stochastique

2.2 – Méthodes avanceées de gradient stochastique (Optimiseurs)

2.3 – TD

Méthodes avancées de gradient stochastique

Méthodes avancées de gradient stochastique / "Optimiseurs"
Pour simplifier, on considère $\alpha_k = \eta > 0$ (pas / *learning rate* constant) et $|S_k| = 1$ (gradient stochastique basique) :

$$w_{k+1} = w_k - \eta \nabla f_{i_k}(w_k) \quad (\text{où } g_k = \nabla f_{i_k}(w_k))$$

Formule générale :

$$w_{k+1} = w_k - \eta m_k \oslash v_k \quad \text{avec } \eta > 0,$$

où $\oslash$ est la division par composante :

- $m_k = ((m_k)_1, \ldots, (m_k)_d)^{\mathrm{T}} \in \mathbb{R}^d$: direction du gradient / moment.
- $v_k = ((v_k)_1, \ldots, (v_k)_d)^{\mathrm{T}} \in \mathbb{R}^d$: vecteur de normalisation selon les coordonnées.

Méthodes avancées de gradient stochastique

1. Optimiseurs avec momentum. Pour $v_k = \mathbf{1}$ (vecteur de 1):

$$w_{k+1} = w_k - \eta m_k \oslash v_k = w_k - \eta m_k.$$

(1.1) SGD avec momentum (*"Heavy ball"*)

$$m_k = \beta_1 m_{k-1} + g_k = \beta_1(\beta_1 m_{k-2} + g_{k-1}) + g_k = g_k + \beta_1 g_{k-1} + \beta_1^2 g_{k-2} + \dots$$

avec $m_{-1} = 0$ et $\beta_1 \in [0, 1)$.

- Si $\beta_1 = 0 \implies$ Pas de momentum (gradient stochastique classique).
- **Valeur typique :** $\beta_1 = 0.9$.
- Très efficace sur les problèmes quadratiques (ex : régression linéaire).
- Ne converge pas en général au-delà sans ajustement.

Méthodes avancées de gradient stochastique

(1.2) *Nesterov-style momentum ("Gradient accéléré" / Méthode de Nesterov)*

$$m_k = \beta_1 m_{k-1} + \nabla f_{i_k}(w_k + \beta_1 m_{k-1})$$
$$w_{k+1} = w_k - \eta m_k$$

Note : Le gradient stochastique est calculé après un "pas de momentum" $(w_k + \beta_1 m_{k-1})$.

En version batch $(\nabla f_{i_k}(w_k) \to \nabla f(w_k))$:

- Offre une meilleure vitesse de convergence optimale sur les problèmes convexes.
- Sensible à la valeur de β_1 (et de η).

Méthodes avancées de gradient stochastique

2. Méthodes à normalisation diagonale (*diagonal scaling*)
On pose $m_k = \nabla f_{i_k}(w_k) \equiv g_k$ et $v_k \neq (1, \ldots, 1)^{\mathrm{T}}$.

But : Adapter le taux d'apprentissage (*learning rate*) à chaque coordonnée au fur et à mesure des itérations.

- **RMSprop (2012) :** $\forall j = 1, \ldots, d$, $(v_k)_j = \sqrt{\beta_2 (v_{k-1})_j^2 + (1 - \beta_2)(g_k)_j^2}$ avec $\beta_2 = 0.9$ (donne plus de poids au passé récent / perte).
- **AdaGrad (2011) :** Normalisation à l'historique $\forall j = 1, \ldots, d$, $(v_k)_j = \sqrt{(v_{k-1})_j^2 + (g_k)_j^2}$ (poids égal du passé et de l'itération courante).

Méthodes avancées de gradient stochastique

(2.1) Adam (Kingma & Ba, 2015)

Note : Plus de 240 000 citations. Adopté pour son efficacité pratique (en apprentissage profond).

$$w_{k+1} = w_k - \eta m_k \oslash v_k$$

$$m_k = \frac{\beta_1 m_{k-1} + (1-\beta_1) g_k}{1-\beta_1^k}$$

$$[v_k]_j = \sqrt{\frac{\beta_2 [v_{k-1}]_j^2 + (1-\beta_2)[g_k]_j^2}{1-\beta_2^k}}$$

avec une moyenne géométrique des gradients, et des valeurs typiques $\beta_1 = 0.9$ et $\beta_2 = 0.999$.
(Kingma & Ba, 2015) : Preuve de convergence dans le cas convexe.

- En 2018 : Contre-exemple publié (la preuve comporte une faille).
- Adam (et ses variantes comme "AdamW") sont les optimiseurs les plus utilisés de nos jours, notamment en TAL / NLP.
- Depuis fin 2024, les méthodes combinant Adam et des optimiseurs dédiés à certains types de paramètres gagnent en popularité.

Méthodes avancées de gradient stochastique (cont.)

(2.2) Algorithm Muon (pour les paramètres sous forme de matrice $\theta \in \mathbb{R}^{d \times d}$)

Algorithm 4 Muon

Require: Learning rate η, momentum μ

1: Initialize $B_0 \leftarrow 0$
2: **for** $t = 1, \ldots$ **do**
3: Compute gradient $G_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_{t-1})$ ▷ gradient stochastique
4: $B_t \leftarrow \mu B_{t-1} + G_t$
5: $O_t \leftarrow \text{NewtonSchulz5}(B_t)$ ▷ Orthogonalisation de B_t
6: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
7: **end for**
8: **return** θ_t

Bilan

- SGD remplace le gradient calculé sur toutes les observations par un estimateur calculé sur une seule. L'espérance de cet estimateur conditionnelle peut être égale au gradient complet (sous l'hypothèse sur l'échantilonnement).
- Mini-batch SGD: compromis entre coût par itération, bruit du gradient et parallélisation.
- Les optimiseurs ajoutent de la mémoire (momentum) et/ou une adaptation coordonnée par coordonnée du pas d'apprentissage.

Plan du cours

1.1 – Descente de gradient

1.2 – Exercices 1

2.1 – Gradient stochastique

2.2 – Méthodes avanceées de gradient stochastique (Optimiseurs)

2.3 – TD

Thank You !

References

Li, H., Xu, Z., Taylor, G., Studer, C., and Goldstein, T. (2018). Visualizing the loss landscape of neural nets. *Advances in neural information processing systems*, 31.

Wright, J. and Ma, Y. (2022). *High-dimensional data analysis with low-dimensional models: Principles, computation, and applications*. Cambridge University Press.